

1 - ¿Qué es el aprendizaje no supervisado?

Introducción al aprendizaje no supervisado: definición, tipos de problemas, ejemplos industriales y flujo típico de trabajo en Data Science.

Introducción al Aprendizaje No Supervisado

Imagina que tienes un conjunto de datos sin etiquetas ni categorías definidas, como registros de clientes, imágenes o transacciones de compra, y quieres descubrir patrones ocultos o estructuras que te ayuden a entender mejor ese conjunto. ¿Cómo podrías hacerlo? Aquí es donde entra en juego el aprendizaje no supervisado, una rama fundamental del aprendizaje automático que permite encontrar relaciones y agrupaciones en datos sin necesidad de respuestas previas.

En esta unidad, exploraremos qué es el aprendizaje no supervisado, cómo se diferencia del aprendizaje supervisado, y cuáles son sus principales tipos de problemas: clustering, reducción de dimensionalidad y reglas de asociación. Además, veremos ejemplos concretos de su aplicación en industrias como retail, tecnología y analítica de negocios, y entenderemos el flujo típico de trabajo para abordar estos problemas.

Al finalizar, serás capaz de definir claramente el aprendizaje no supervisado y reconocer sus usos más comunes en Data Science, sentando las bases para implementar algoritmos específicos en las unidades siguientes.



¿Qué es el Aprendizaje No Supervisado?

El **aprendizaje no supervisado** es un conjunto de técnicas de machine learning que buscan identificar estructuras, patrones o relaciones en datos que no cuentan con etiquetas o respuestas conocidas. A diferencia del aprendizaje supervisado, donde el modelo aprende a partir de ejemplos con etiquetas (por ejemplo, clasificar correos como spam o no spam), en el aprendizaje no supervisado el objetivo es descubrir información oculta sin guía explícita.

Diferencias clave con el aprendizaje supervisado

Característica	Aprendizaje Supervisado	Aprendizaje No Supervisado
Datos de entrada	Datos con etiquetas o respuestas	Datos sin etiquetas
Objetivo	Predecir o clasificar	Encontrar patrones o estructuras
Ejemplos de problemas	Clasificación, regresión	Clustering, reducción de dimensionalidad, reglas de asociación

Tipos principales de problemas en aprendizaje no supervisado

1. **Clustering (agrupamiento):**

- Busca agrupar datos similares en clusters o grupos.
- Ejemplo: segmentar clientes según comportamiento de compra.

2. **Reducción de dimensionalidad:**

- Simplifica datos complejos con muchas variables a representaciones más manejables.
- Ejemplo: usar PCA para visualizar datos en 2D o 3D.

3. **Reglas de asociación:**

- Encuentra relaciones frecuentes entre variables.
- Ejemplo: identificar productos que se compran juntos en retail.

***Nota:** Estas categorías serán exploradas en detalle en unidades posteriores, donde implementaremos algoritmos específicos y analizaremos resultados.*

Ejemplos de aplicación en la industria

- **Retail y E-commerce:** Segmentación de clientes para campañas personalizadas, análisis de cesta de la compra con reglas de asociación.
- **Tecnología y Big Data:** Detección de anomalías en redes, agrupamiento de documentos o imágenes.
- **Analítica de negocios:** Reducción de variables para simplificar reportes y visualizaciones.

Estos ejemplos muestran cómo el aprendizaje no supervisado ayuda a extraer valor de datos sin necesidad de etiquetas previas, facilitando la toma de decisiones basada en patrones reales.

Flujo típico de trabajo en aprendizaje no supervisado

1. **Recolección y preparación de datos:** Limpieza, selección y escalado de variables.
2. **Selección del método adecuado:** Según el problema y tipo de datos.
3. **Aplicación del algoritmo:** Ejecución y ajuste de parámetros.
4. **Evaluación y validación:** Uso de métricas específicas para medir calidad de agrupamientos o representaciones.
5. **Interpretación y uso de resultados:** Integración en procesos de negocio o análisis posteriores.

Este flujo será la base para las prácticas y análisis en las siguientes unidades.

En el contexto actual de Data Science, el aprendizaje no supervisado es una herramienta esencial para explorar y entender grandes volúmenes de datos sin etiquetas, que son comunes en muchas industrias. Por ejemplo, en retail, conocer cómo se agrupan los clientes o qué productos se compran juntos puede mejorar significativamente las estrategias de marketing y ventas.

Además, la reducción de dimensionalidad permite a los científicos de datos visualizar y manejar datos complejos, facilitando la detección de patrones y la comunicación de resultados a equipos no técnicos.

Este conocimiento es fundamental para cualquier profesional de Data Science que busque aplicar técnicas avanzadas y obtener insights valiosos a partir de datos reales, preparando el terreno para implementar algoritmos como K-Means o PCA en las próximas unidades.

2 - Teoría de reglas de asociación y métricas (support, confidence, lift)

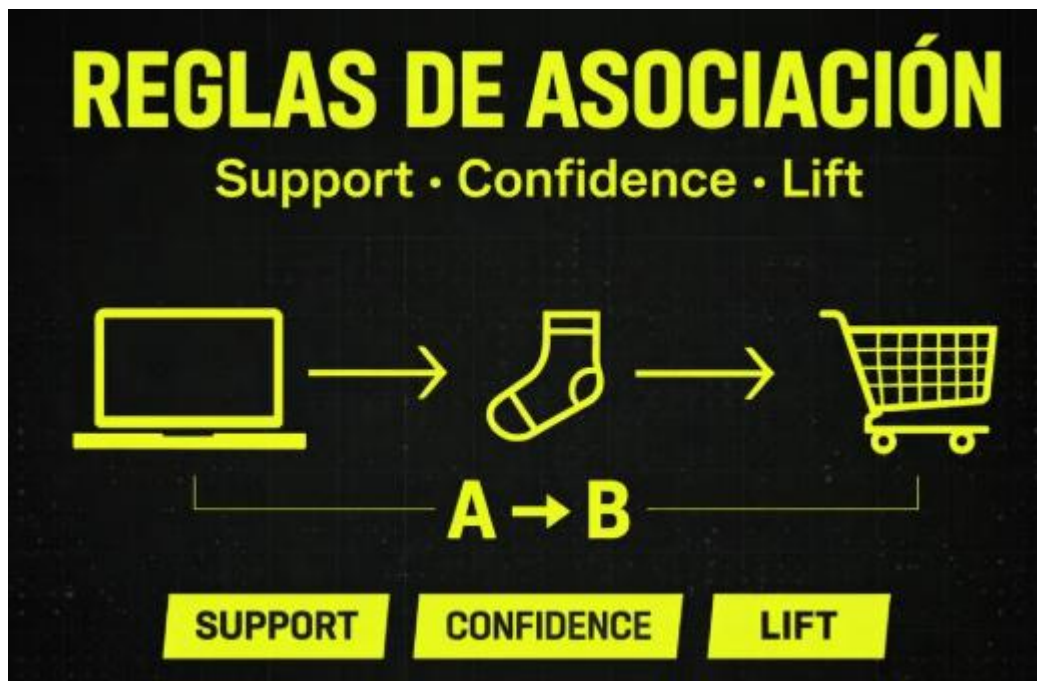
Introducción conceptual a las reglas de asociación, explicando los algoritmos Apriori y FP-Growth, y las métricas clave support, confidence y lift, con énfasis en su aplicación práctica en retail y e-commerce.

¿Alguna vez te has preguntado cómo las tiendas en línea saben qué productos recomendarte juntos? Por ejemplo, ¿por qué al comprar una laptop también te sugieren una funda o un mouse? Esto es posible gracias a las **reglas de asociación**, una técnica fundamental en análisis de datos para descubrir patrones frecuentes en grandes conjuntos de transacciones.

En esta unidad, exploraremos los conceptos básicos detrás de estas reglas, los algoritmos más usados para extraerlas —Apriori y FP-Growth— y las métricas que nos permiten evaluar su relevancia: **support**, **confidence** y **lift**. Al finalizar, comprenderás cómo y cuándo aplicar estas reglas en contextos reales de retail y e-commerce para mejorar la toma de decisiones y las recomendaciones personalizadas.

Reglas de Asociación: Conceptos Fundamentales

En esta unidad, exploraremos los conceptos básicos detrás de estas reglas, los algoritmos más usados para extraerlas —Apriori y FP-Growth— y las métricas que nos permiten evaluar su relevancia: **support**, **confidence** y **lift**. Al finalizar, comprenderás cómo y cuándo aplicar estas reglas en contextos reales de retail y e-commerce para mejorar la toma de decisiones y las recomendaciones personalizadas.



Apriori

El algoritmo **Apriori** es uno de los métodos clásicos para encontrar conjuntos frecuentes de ítems.

Funciona generando candidatos de conjuntos de ítems y evaluando su frecuencia en las transacciones, descartando aquellos que no cumplen un umbral mínimo de frecuencia (support).

- **Ventajas:** Intuitivo y fácil de implementar.
- **Desventajas:** Puede ser computacionalmente costoso en grandes bases de datos debido a la generación masiva de candidatos.

FP-Growth

FP-Growth (Frequent Pattern Growth) es un algoritmo más eficiente que evita la generación explícita de candidatos. Construye una estructura llamada árbol FP que compacta la información de las transacciones y extrae patrones frecuentes directamente.

- **Ventajas:** Más rápido y escalable que Apriori en grandes volúmenes de datos.
- **Desventajas:** Implementación más compleja.

Métricas Clave para Evaluar Reglas

Para evaluar la calidad y utilidad de una regla de asociación, usamos tres métricas principales:

Métrica	Definición	Interpretación
Support (Soporte)	Proporción de transacciones que contienen ambos ítems A y B: $\text{support}(A \rightarrow B) = P(A \cap B)$	Indica la frecuencia con la que ocurre la regla en el conjunto de datos.
Confidence (Confianza)	Probabilidad de que B ocurra dado que A ocurrió: $\text{confidence}(A \rightarrow B) = P(B A)$	$\text{confidence}(A \rightarrow B) = \frac{\text{support}(A \cap B)}{\text{support}(A)}$
Lift	Relación entre la confianza y la frecuencia esperada de B: $\text{lift}(A \rightarrow B) = \frac{\text{confidence}(A \rightarrow B)}{\text{support}(B)}$	Indica la fuerza de la asociación; valores >1 sugieren una relación positiva entre A y B.

***Nota:** Una regla con alto support y confidence es frecuente y confiable, pero el lift nos ayuda a entender si la asociación es significativa o simplemente por azar.*

¿Cuándo usar reglas de asociación en retail?

- Para descubrir productos que se compran juntos y diseñar promociones cruzadas.
- Para optimizar la disposición de productos en tiendas físicas o virtuales.
- Para personalizar recomendaciones en plataformas de e-commerce.

Estas reglas permiten transformar datos transaccionales en insights accionables que mejoran la experiencia del cliente y aumentan las ventas.

En el contexto de retail y e-commerce, las reglas de asociación son herramientas poderosas para analizar grandes volúmenes de datos de ventas y descubrir patrones ocultos. Por ejemplo, un minorista puede usar Apriori o FP-Growth para identificar combinaciones frecuentes de productos y luego diseñar campañas de marketing dirigidas o bundles atractivos.

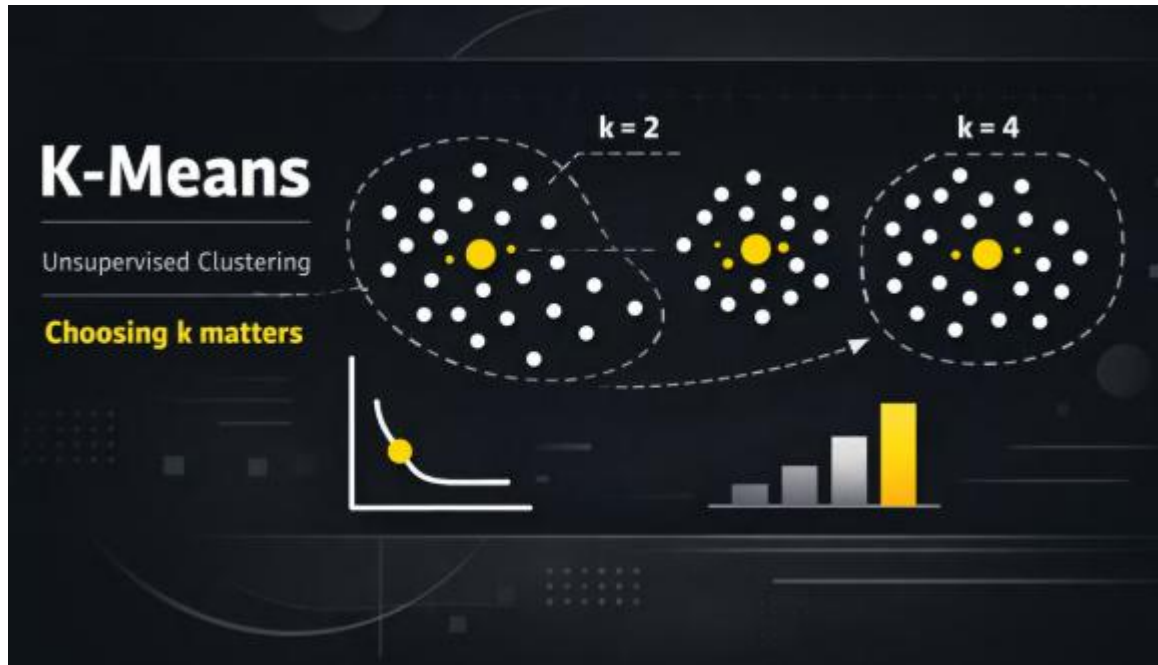
Además, entender las métricas support, confidence y lift es crucial para seleccionar reglas que sean no solo frecuentes, sino también relevantes y útiles para la toma de decisiones. Por ejemplo, una regla con alto support pero bajo lift puede no ser interesante, ya que la asociación podría ser casual.

Este conocimiento prepara el terreno para la siguiente unidad, donde implementaremos estos algoritmos en Python y aplicaremos estas métricas para evaluar y seleccionar las mejores reglas en casos reales de retail.

3 - Teoría de K-Means y elección de k

Introducción al algoritmo K-Means, su funcionamiento, convergencia, limitaciones y métodos prácticos para seleccionar el número óptimo de clusters (k).

¿Cómo agrupar datos sin etiquetas? Descubre K-Means



Imagina que tienes un conjunto de datos sin etiquetas y quieres encontrar grupos naturales dentro de ellos. ¿Cómo podrías hacerlo? Aquí es donde entra en juego el algoritmo **K-Means**, una técnica fundamental en aprendizaje no supervisado que permite identificar patrones y segmentar datos en grupos similares.

En esta unidad, exploraremos cómo funciona K-Means, desde su algoritmo básico hasta los desafíos que presenta, como la elección del número de clusters, y cómo la inicialización puede afectar los resultados. Al finalizar, estarás preparado para implementar K-Means con confianza y seleccionar el parámetro clave k de manera informada, habilidades esenciales para cualquier científico de datos que trabaje con segmentación y análisis exploratorio.

Algoritmo K-Means: ¿Qué es y cómo funciona?

K-Means es un algoritmo de **partición** que divide un conjunto de datos en k grupos (clusters) basándose en la similitud de sus características. El objetivo es minimizar la suma de las distancias entre cada punto y el centroide (promedio) de su cluster asignado.

Pasos del algoritmo:

1. **Inicialización:** Se eligen k centroides iniciales, que pueden ser seleccionados aleatoriamente o mediante métodos como `k-means++` para mejorar la convergencia.
2. **Asignación:** Cada punto se asigna al cluster cuyo centroide esté más cercano, usando una métrica de distancia (por ejemplo, Euclidiana).
3. **Actualización:** Se recalculan los centroides como la media de los puntos asignados a cada cluster.
4. **Repetición:** Se repiten los pasos 2 y 3 hasta que las asignaciones no cambien o se alcance un número máximo de iteraciones.

***Nota:** La métrica de distancia es crucial para determinar la similitud. En esta unidad reforzamos las métricas Euclidiana, Manhattan y Coseno, que ya conoces, y que afectan directamente la forma en que se agrupan los datos.*

Convergencia y efectos de la inicialización

- K-Means siempre converge a un mínimo local, pero no necesariamente al óptimo global.
- La **inicialización** de los centroides puede afectar la calidad y velocidad de convergencia.
- Métodos como `k-means++` ayudan a elegir centroides iniciales más representativos, reduciendo la probabilidad de resultados pobres.

Problemas comunes

- **Outliers:** Pueden distorsionar los centroides y afectar la agrupación.
- **Formas no esféricas:** K-Means asume clusters convexos y esféricos; no funciona bien con clusters de formas arbitrarias o tamaños muy diferentes.

Métodos para elegir el número de clusters k

Elegir un k adecuado es fundamental para obtener clusters significativos. Dos métodos prácticos son:

1. Método del codo (Elbow Method)

El método del codo es una técnica comúnmente utilizada para determinar el número óptimo de clusters en un análisis de agrupamiento, especialmente cuando se utiliza el algoritmo K-means. La idea principal es encontrar un equilibrio entre la cantidad de clusters y la calidad de la agrupación, de manera que el número elegido no sea ni demasiado pequeño (subajuste) ni demasiado grande (sobreajuste).

El procedimiento consiste en lo siguiente:

- Para diferentes valores de k , donde k representa el número de clusters que queremos probar, se calcula la suma de errores cuadráticos dentro del cluster, conocida en inglés como Within-Cluster Sum of Squares (WCSS). Este valor mide la variabilidad interna de cada cluster sumando las distancias al cuadrado entre cada punto y el centroide de su cluster correspondiente. Un WCSS más bajo indica clusters más compactos y homogéneos.
- Luego, se grafica el valor de WCSS en función de k . Esta gráfica generalmente muestra una disminución del WCSS a medida que aumentamos el número de clusters porque agrupar en más clusters reduce la distancia interna a los centroides.
- El objetivo es identificar un punto en esta curva donde la tasa de disminución del WCSS empieza a reducirse notablemente, formando un “codo”. Este codo representa el punto a partir del cual agregar más clusters no mejora significativamente la calidad de la agrupación. Por lo tanto, el valor de k correspondiente a este codo se considera como el número óptimo de clusters para el conjunto de datos analizado.

En resumen, el método del codo permite seleccionar un valor de k que balancea bien la complejidad del modelo con la calidad de la agrupación, evitando tanto un número insuficiente como excesivo de clusters.

2. Coeficiente silhouette

- El coeficiente silhouette es una métrica utilizada en análisis de clusters para evaluar qué tan bien se ha realizado la agrupación de los datos. Esta medida cuantifica qué tan similar es un punto dentro de su propio cluster en comparación con los puntos de otros clusters, proporcionando así una indicación de la calidad y coherencia de los grupos formados.

Para cada punto, el coeficiente silhouette se calcula considerando dos valores principales: la distancia promedio entre ese punto y todos los demás puntos dentro del mismo cluster (conocida como la cohesión del cluster) y la distancia promedio entre ese punto y todos los puntos del cluster más cercano al que no pertenece (conocida como separación entre clusters). La fórmula combina estos dos aspectos para producir un valor que oscila entre -1 y 1. Un valor cercano a 1 indica que el punto está muy bien asignado a su cluster, ya que está mucho más cerca de los puntos dentro de su grupo que de los puntos en otros clusters. Por el contrario, un valor cercano a -1 sugiere que el punto podría estar mal asignado y sería más apropiado en otro cluster.

Para evaluar la calidad general del agrupamiento, se calcula el coeficiente silhouette promedio considerando todos los puntos del conjunto de datos. Este promedio se utiliza para comparar diferentes

configuraciones del hiperparámetro k , que representa el número de clusters en la agrupación. El valor óptimo de k es aquel que maximiza el coeficiente silhouette promedio, indicando que los clusters formados son los más definidos y separados posibles bajo ese número de grupos. De esta manera, el coeficiente silhouette no solo ayuda a validar la calidad de un agrupamiento, sino también a determinar la cantidad más adecuada de clusters para segmentar eficazmente los datos.

- Mide qué tan similar es un punto a su propio cluster comparado con otros clusters.
- El valor oscila entre -1 y 1; valores cercanos a 1 indican clusters bien definidos.
- Se calcula el promedio del coeficiente silhouette para diferentes k y se elige el que maximiza este valor.
- Es útil para evaluar la coherencia interna del agrupamiento sin necesidad de etiquetas.
- Puede ayudar a identificar outliers que tengan coeficientes cerca de -1.

Ejemplo: En un análisis de segmentación de clientes para e-commerce, elegir un k adecuado permite identificar grupos con comportamientos de compra similares, optimizando campañas de marketing personalizadas.

Aplicación práctica y relevancia en la industria

En sectores como retail, e-commerce y analítica de negocios, K-Means es una herramienta clave para segmentar clientes, detectar patrones de comportamiento y optimizar recursos.

Por ejemplo, un retailer puede usar K-Means para agrupar productos según características de venta y stock, facilitando la gestión de inventarios y promociones.

Un ejemplo real de segmentación de clientes es el caso de una cadena de supermercados que utiliza K-Means para dividir a sus clientes en grupos según su frecuencia de compra, monto gastado y preferencia por categorías de productos. Esto les permite diseñar campañas de marketing personalizadas, mejorar la experiencia del cliente y aumentar la fidelización.

Además, la correcta elección de k evita tanto la sobresegmentación (demasiados clusters que complican la interpretación) como la subsegmentación (pocos clusters que ocultan diferencias importantes).

Este conocimiento te prepara para implementar K-Means en tus proyectos, interpretar resultados y ajustar parámetros para obtener insights valiosos y accionables. También es común su aplicación en análisis de

imágenes para la segmentación de regiones y en sistemas de recomendación personalizados.

En la industria financiera, K-Means ayuda a identificar grupos de clientes con perfiles de riesgo similares, mejorando la gestión de cartera y la detección de fraudes.

Por último, la combinación de K-Means con otras técnicas de Machine Learning puede potenciar modelos predictivos más robustos.

4 - Clustering jerárquico y DBSCAN: conceptos y cuándo usarlos

Introducción a los métodos de clustering jerárquico y DBSCAN, sus características, parámetros clave, ventajas y limitaciones, con ejemplos visuales para facilitar la comprensión y comparación con otros métodos de clustering.

¿Alguna vez te has preguntado cómo agrupar datos sin tener etiquetas previas? En el mundo real, desde segmentar clientes en e-commerce hasta identificar patrones en datos geoespaciales, el clustering es una herramienta fundamental. En esta unidad, exploraremos dos métodos avanzados de clustering: el jerárquico y DBSCAN. Aprenderás no solo qué son y cómo funcionan, sino también cuándo es mejor usar cada uno, entendiendo sus parámetros clave y visualizando sus resultados para tomar decisiones informadas en proyectos de ciencia de datos.

Clustering Jerárquico

El **clustering jerárquico** es un método que construye una jerarquía de clusters, representada comúnmente mediante un **dendrograma**. Existen dos enfoques principales:

- **Aglomerativo**: comienza con cada punto como un cluster individual y va fusionándolos progresivamente.
- **Divisivo**: inicia con todos los puntos en un solo cluster y los divide sucesivamente.

El proceso aglomerativo es el más utilizado en la práctica. Para fusionar o dividir clusters se emplean diferentes criterios, conocidos como **métricas de enlace**, tales como:

- **Enlace simple (single linkage)**: distancia mínima entre puntos de dos clusters.
- **Enlace completo (complete linkage)**: distancia máxima entre puntos de dos clusters.
- **Enlace promedio (average linkage)**: promedio de las distancias entre pares de puntos de dos clusters.

El dendrograma resultante permite visualizar la estructura jerárquica y elegir el número de clusters mediante cortes horizontales. El **clustering jerárquico** es una técnica de análisis de datos que permite agrupar un conjunto de objetos o puntos en grupos llamados clusters, de manera que los elementos dentro

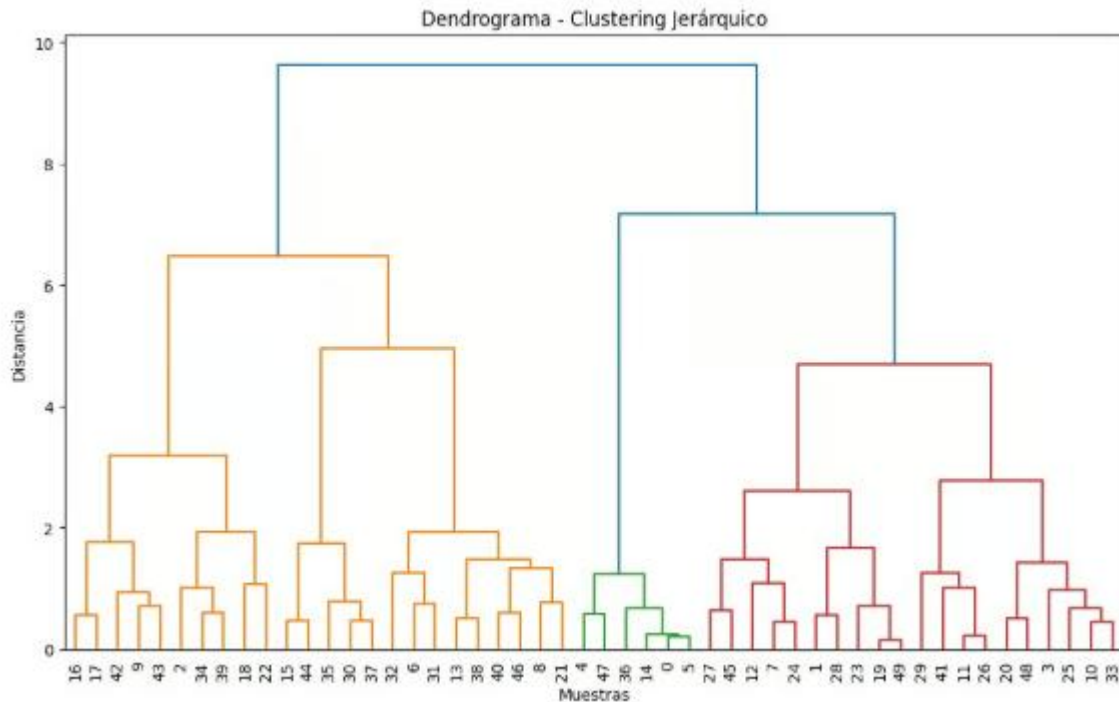
de un mismo cluster sean más similares entre sí que con los de otros clusters. A diferencia de otros métodos de clustering, el clustering jerárquico no necesita definir previamente el número de clusters, sino que construye una estructura en forma de árbol llamada **jerarquía**, que muestra cómo los clusters se combinan o dividen en diferentes niveles de similitud.

El **dendrograma**, que es un diagrama en forma de árbol. En un dendrograma, cada hoja representa un punto o dato individual, y los nodos en el interior del árbol muestran cómo esos puntos o pequeños clusters se unen para formar clusters más grandes. La altura o nivel en el dendrograma donde dos clusters se unen indica el grado de disimilitud o distancia entre ellos: cuanto más abajo se unan, más similares son. Esto

ayuda a visualizar y entender la estructura y las relaciones de agrupamiento en los datos.

```
# DENDOGRAMA COMPLETO EN UNA CELDA
# Importamos librerías necesarias
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# -----
# 1) Crear o cargar un dataset
#     (Acá genero un dataset de ejemplo con variables continuas)
# -----
np.random.seed(42)
data = pd.DataFrame({
    "Ingresos": np.random.randint(20000, 150000, 50),
    "Gasto_Mensual": np.random.randint(5000, 40000, 50),
    "Edad": np.random.randint(18, 70, 50)})
print("Primeras filas del dataset:")
print(data.head())
# -----
# 2) Estandarización
# -----
scaler = StandardScaler()
data_scaled = scaler.fit_transform(data)
# -----
# 3) Cálculo de linkage para el dendrograma
#     Método Ward minimiza la varianza dentro de los clusters
# -----
Z = linkage(data_scaled, method='ward')
# -----
# 4) Graficar dendrograma
# -----
plt.figure(figsize=(12, 7))
plt.title("Dendrograma - Clustering Jerárquico")
plt.xlabel("Muestras")
plt.ylabel("Distancia")
dendrogram(Z, leaf_rotation=90, # Rota labels para mejor lectura
           leaf_font_size=10 # Tamaño de texto)
plt.show()
```



Existen dos enfoques principales para construir esta jerarquía:

- **Clustering aglomerativo:** Este método comienza con una situación inicial donde cada punto de datos es considerado como un cluster individual separado. El algoritmo busca iterativamente los dos clusters más parecidos o cercanos para fusionarlos en un nuevo cluster más grande. Este proceso se repite hasta que todos los puntos se hayan combinado en un único cluster que contiene todos los datos. Por ejemplo, si tienes cinco datos, primero hay cinco clusters individuales; luego se combinan los dos más similares para formar cuatro clusters; después otros dos se unen para formar tres clusters; y así sucesivamente. La ventaja del enfoque aglomerativo es que es intuitivo y permite explorar diferentes niveles de agrupamiento, porque se puede cortar el dendrograma en diferentes alturas para obtener diferentes números de clusters según el interés del análisis.
- **Clustering divisivo:** En este método, la aproximación es la inversa al aglomerativo. Se parte de un solo cluster que contiene todo el conjunto de datos. A partir de allí, el algoritmo divide progresivamente ese cluster grande en clusters más pequeños. En cada paso, se separan los grupos que son menos cohesivos o que tienen mayor disimilitud interna, dividiendo hasta que se alcanzan clusters con características o tamaños deseados. Esta técnica

Parámetro clave: Linkage

El método de *linkage* determina cómo se mide la distancia entre clusters para decidir cuáles fusionar:

- **Single linkage:** distancia mínima entre puntos de dos clusters.
- **Complete linkage:** distancia máxima entre puntos.
- **Average linkage:** promedio de todas las distancias.

Cada método afecta la forma y tamaño de los clusters resultantes.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

DBSCAN es un algoritmo basado en densidad que identifica clusters como regiones densas separadas por regiones menos densas, y detecta puntos aislados como **ruido**.

Parámetros clave:

- **eps (epsilon):** radio máximo para considerar vecinos.
- **min_samples:** número mínimo de puntos para formar un cluster denso.

DBSCAN es especialmente útil para detectar clusters de forma arbitraria y manejar ruido, a diferencia de métodos que asumen formas convexas.

```
# -----
# 1) Imports
# -----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons, make_blobs
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score

# -----
# 2) Generar dataset de ejemplo
# -----
# DBSCAN funciona bien con formas arbitrarias (ej. lunas), por eso creamos:
# - make_moons: dos semicírculos (no convexos)
# - make_blobs: algunos centros densos
# Además agregamos puntos aleatorios como ruido. np.random.seed(42)
# crear "lunas" (formas no convexas)
X1, _ = make_moons(n_samples=300, noise=0.08)
# crear blobs (clusters densos)
X2, _ = make_blobs(n_samples=150, centers=[(3,3), (6,-1)],
cluster_std=[0.3,0.6])
# puntos de ruido dispersos noise = np.random.uniform(low=-3, high=8, size=(60,2))
# concatenar todo X = np.vstack([X1, X2, noise])
```

```

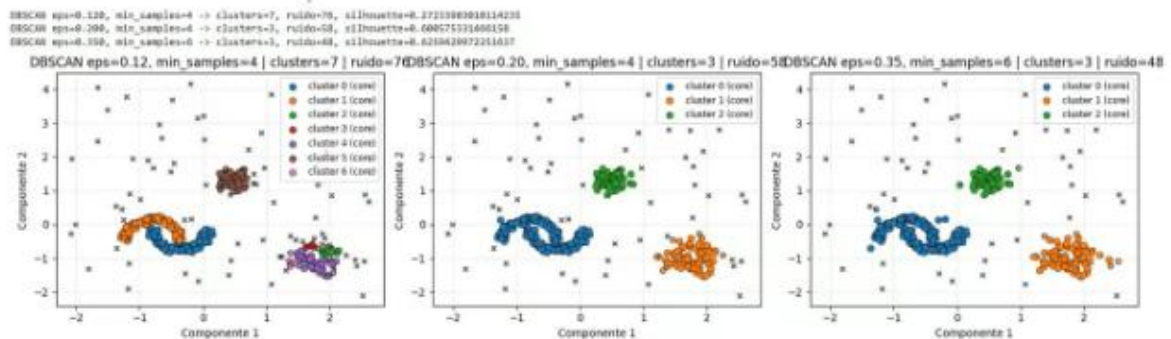
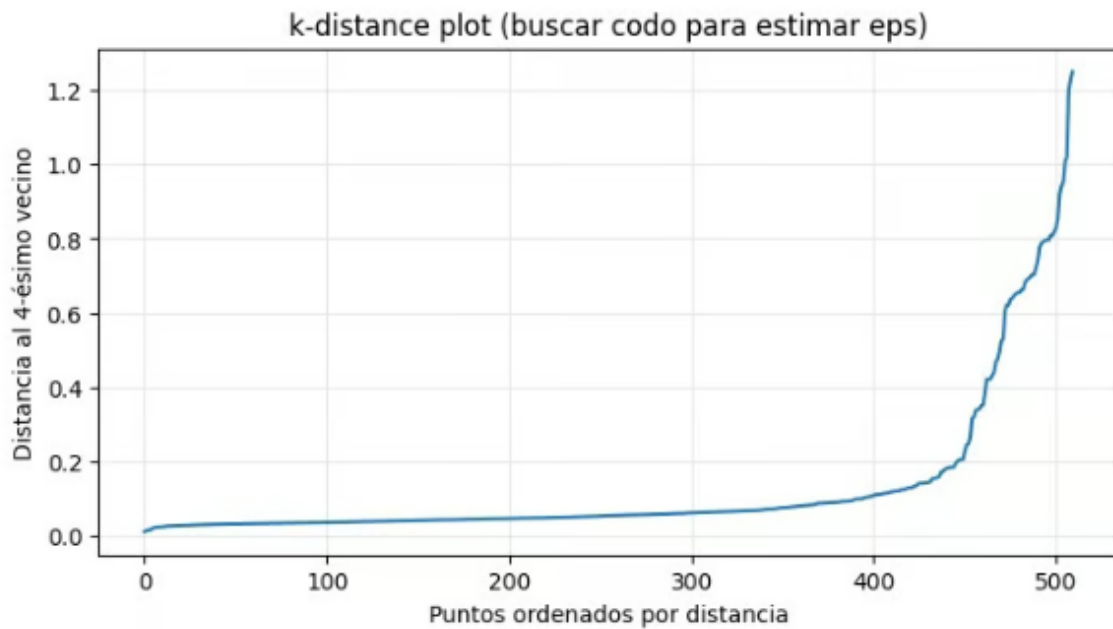
# estandarizar para que eps sea interpretable en escala similarscaler =
StandardScaler()
X_scaled = scaler.fit_transform(X)
# breve vista del dataset
print("Shape del dataset:", X_scaled.shape)
print("Primeras filas (X_scaled):")
print(pd.DataFrame(X_scaled, columns=["x1", "x2"]).head())
# -----
# 3) k-distance plot (curva de codo) para ayudar a elegir eps
# -----
# Teoría: para elegir eps, se suele calcular la distancia al k-ésimo vecino
(k = min_samples)
# y buscar un "codo" en la curva de distancias ordenadas. Ese codo es un
buen candidato para eps.k = 4 # común usar min_samples-1 o min_samplesnbrs
= NearestNeighbors(n_neighbors=k).fit(X_scaled)
distances, _ = nbrs.kneighbors(X_scaled)
# distancia al k-ésimo vecino para cada puntok_distances =
np.sort(distances[:, -1])
plt.figure(figsize=(8,4))
plt.plot(k_distances)
plt.ylabel(f"Distancia al {k}-ésimo vecino")
plt.xlabel("Puntos ordenados por distancia")
plt.title("k-distance plot (buscar codo para estimar eps)")
plt.grid(alpha=0.3)
plt.show()
# -----
# 4) Función auxiliar para ejecutar DBSCAN y graficar
# -----
def run_and_plot_dbscan(X, eps, min_samples, ax=None):    """    Ejecuta
DBSCAN y grafica resultado.    - X : array (n_samples, 2)    - eps : radio
máximo para vecinos (epsilon)    - min_samples : puntos mínimos para formar
un cluster denso    """    # Ajuste del algoritmo    db = DBSCAN(eps=eps,
min_samples=min_samples)    labels = db.fit_predict(X)    # label = -1 -->
ruido    unique_labels = set(labels)    # Conteos básicos    n_clusters
= len([lab for lab in unique_labels if lab != -1])    n_noise =
list(labels).count(-1)    # Silhouette sólo si hay al menos 2 clusters
sil = None    if n_clusters > 1:        try:            sil =
silhouette_score(X, labels)        except Exception:            sil = None
# Informar parámetros y métricas clave    print(f"DBSCAN eps={eps:.3f},
min_samples={min_samples} -> clusters={n_clusters}, ruido={n_noise},
silhouette={sil}")    # Preparar colores para plot    # label -1 será
el color negro para marcar ruido    colors = plt.cm.tab10.colors    # paleta
# plot    if ax is None:        fig, ax = plt.subplots(figsize=(6,5))    #
marcar puntos core vs border vs noise:    core_samples_mask =
np.zeros_like(labels, dtype=bool)    # scikit-learn guarda índices de core
samples en db.core_sample_indices_ si los hay    if hasattr(db,
'core_sample_indices_'):        core_samples_mask[db.core_sample_indices_]
= True    for lab in unique_labels:        if lab == -1:            #
ruido            col = (0,0,0)    # negro            marker = 'x'
size = 30            label_name = "ruido (-1)"        else:            col
= colors[lab % len(colors)]            marker = 'o'            size = 40
label_name = f"cluster {lab}"        class_member_mask = (labels ==
lab)    # core points        xy_core = X[class_member_mask &
core_samples_mask]    # border points        xy_border =
X[class_member_mask & ~core_samples_mask]        if len(xy_core) >
0:            ax.scatter(xy_core[:,0], xy_core[:,1], s=size, marker=marker,
edgecolor='k', linewidth=0.4, facecolor=col, label=label_name + " (core)"
if lab != -1 else label_name)        if len(xy_border) > 0:            #
border points plot con menor alpha            ax.scatter(xy_border[:,0],
xy_border[:,1], s=size*0.6, marker=marker, facecolor=col, edgecolor='k',
alpha=0.7)            ax.set_title(f"DBSCAN eps={eps:.2f},

```

```

min_samples={min_samples} | clusters={n_clusters} | ruido={n_noise}")
ax.set_xlabel("Componente 1")    ax.set_ylabel("Componente 2")
ax.legend(loc='best', fontsize='small')    ax.grid(alpha=0.25)
# -----
# 5) Ejecutar DBSCAN con varios pares (eps, min_samples) para comparar
# -----
# Aquí mostramos cómo varían resultados según eps y min_samples.
# - eps pequeño -> más puntos considerados aislados (ruido)
# - eps grande -> se fusionan clusters; puede que todo quede en 1 cluster
# - min_samples grande -> requiere mayor densidad para formar cluster (más
ruido)
configs = [    (0.12, 4), # eps pequeño: muchos puntos en ruido    (0.20,
4), # eps intermedio: suele detectar lunas y blobs    (0.35, 6) # eps
más grande + min_samples mayor: clusters más grandes, menos ruido]
fig, axes = plt.subplots(1, len(configs), figsize=(5*len(configs), 4))for
ax, (eps, ms) in zip(axes.flatten(), configs):
run_and_plot_dbscan(X_scaled, eps=eps, min_samples=ms, ax=ax)
plt.tight_layout()
plt.show()
# -----
# 6) Conceptos teóricos ligados al código (resumen)
# -----
# DBSCAN (Density-Based Spatial Clustering of Applications with Noise):
# - Identifica clusters como regiones densas separadas por regiones de baja
densidad.
# - Parámetros:
#   * eps: radio máximo para considerar vecinos. En el código usamos la k-
distance plot para estimarlo.
#   * min_samples: número mínimo de puntos para que una región sea
considerada densa (core point).
# - Tipos de puntos en DBSCAN:
#   * core points: puntos con al menos min_samples en su vecindad de radio
eps.
#   * border points: puntos que están dentro del eps de un core point pero
no tienen min_samples por sí mismos.
#   * noise points (ruido): puntos que no son ni core ni border (label = -1
en scikit-learn).
# - Ventajas observadas en el ejemplo:
#   * Detecta clusters de forma arbitraria (las "lunas" quedan bien
identificadas).
#   * Maneja ruido explícitamente (puntos marcados como -1).
# - Limitaciones:
#   * Elección de eps y min_samples es crítica y depende de la escala de
los datos.
#   * No funciona bien si la densidad de los clusters varía mucho (clusters
con densidades

```



Comparación con otros métodos

Característica	Jerárquico	Particional (e.g., K-Means)	DBSCAN
Forma de clusters	Jerarquía flexible	Convexa, esférica	Arbitraria
Número de clusters	No requiere definir	Requiere definir	No requiere definir
Manejo de ruido	No explícito	No explícito	Sí, detecta ruido
Parámetros clave	Linkage	Número de clusters (k)	eps, min_samples

Nota: La elección del método depende del tipo de datos y del objetivo del análisis.

En la práctica, el clustering jerárquico es útil para explorar la estructura de los datos y decidir el número óptimo de clusters mediante el análisis del dendrograma. Por ejemplo, en retail, puede ayudar a segmentar productos según similitud de ventas o características.

DBSCAN, por otro lado, es ideal para detectar grupos con formas irregulares y manejar ruido, como identificar zonas de alta concentración de clientes en análisis geoespacial para campañas de marketing.

Ambos métodos complementan a los algoritmos particionales como K-Means, ofreciendo alternativas cuando las suposiciones de estos últimos no se cumplen.

En esta unidad, prepararemos el terreno para implementar estos algoritmos en Python, entender sus parámetros y evaluar sus resultados con métricas que veremos en unidades posteriores.

5 - Teoría de PCA y consideraciones prácticas

Introducción conceptual al Análisis de Componentes Principales (PCA), explicando la matemática básica detrás de la covarianza, eigenvectores y eigenvalores, la varianza explicada y las limitaciones del método, para que el estudiante pueda interpretar y aplicar PCA en contextos reales de ciencia de datos.

Introducción a la reducción de dimensionalidad y PCA

Imagina que trabajas con un conjunto de datos con decenas o incluso cientos de variables — por ejemplo, características de clientes en un e-commerce o sensores en un sistema IoT. ¿Cómo podemos simplificar esta información sin perder lo esencial? Aquí es donde entra el **Análisis de Componentes Principales (PCA)**, una técnica fundamental para reducir la dimensionalidad de los datos, facilitando su visualización y análisis.

En esta unidad, exploraremos la teoría detrás de PCA, entendiendo conceptos matemáticos clave como la **covarianza**, los **eigenvectores** y **eigenvalores**, y cómo estos nos permiten identificar las direcciones de máxima varianza en los datos. También aprenderemos a interpretar la **varianza explicada** para decidir cuántos componentes conservar, y discutiremos las limitaciones prácticas de PCA, especialmente su naturaleza lineal.

Este conocimiento te permitirá comprender por qué y cómo PCA transforma tus datos, preparándote para implementarlo y analizar sus resultados en las siguientes unidades.



Conceptos clave para entender PCA

Covarianza: la relación entre variables

La **covarianza** mide cómo dos variables varían juntas. Si dos variables tienden a aumentar o disminuir simultáneamente, su covarianza es positiva; si una aumenta mientras la otra disminuye, es negativa.

Matemáticamente, para dos variables aleatorias (X) y (Y):

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

Donde (E) es la esperanza matemática y (μ_X , μ_Y) son las medias de (X) y (Y).

En PCA, la matriz de covarianza resume las relaciones entre todas las variables, siendo la base para identificar las direcciones de mayor variabilidad.

Eigenvectores y eigenvalores: direcciones y magnitudes

Un **eigenvector** es un vector que, al aplicar una transformación lineal (como la matriz de covarianza), solo cambia en magnitud, no en dirección. El factor por el que cambia la magnitud es el **eigenvalor** correspondiente.

En PCA, los eigenvectores de la matriz de covarianza representan las **componentes principales** — las nuevas direcciones ortogonales sobre las que proyectamos los datos. Los eigenvalores indican la **varianza** que cada componente explica.

- **Eigenvectores:** Se identifican correctamente como las **Componentes Principales** (las nuevas direcciones ortogonales).
- **Eigenvalores:** Se identifican correctamente como la **Varianza** que explica cada componente (la magnitud).
- El ejemplo es muy útil: un eigenvalor alto = mucha variabilidad capturada.

***Ejemplo:** Si un eigenvector tiene un eigenvalor alto, significa que la dirección que representa captura mucha variabilidad de los datos.*

Varianza explicada y selección de componentes

La suma de todos los eigenvalores es la varianza total de los datos. La **varianza explicada** por cada componente es el porcentaje que representa su eigenvalor respecto a la suma total.

Esto nos ayuda a decidir cuántos componentes conservar:

- Conservamos los primeros componentes que expliquen un porcentaje significativo (por ejemplo, 90%) de la varianza.
- Esto reduce dimensionalidad manteniendo la mayor información posible.
- Un gráfico de codo puede ayudar a visualizar el punto donde agregar más componentes aporta poca varianza adicional.
- También se puede usar un umbral fijo para la varianza explicada acumulada, seleccionando el número mínimo de componentes que lo superen.
- En algunos casos, priorizar menos componentes para simplificar el modelo puede ser preferible aunque se pierda algo de varianza.

Varianza explicada y selección de componentes

La varianza es una medida que indica cuánto varían o se dispersan los datos respecto a su promedio. La suma de todos los eigenvalores es la varianza total de los datos. La **varianza explicada** por cada componente es el porcentaje que representa su eigenvalor respecto a la suma total.

Esto nos ayuda a decidir cuántos componentes conservar:

- Conservamos los primeros componentes que expliquen un porcentaje significativo (por ejemplo, 90%) de la varianza.
 - Esto reduce dimensionalidad manteniendo la mayor información posible.
-

Limitaciones de PCA

- **Linealidad:** PCA solo captura relaciones lineales entre variables. Si los datos tienen estructuras no lineales complejas, PCA puede no ser suficiente.
 - **Escalado:** Es sensible a la escala de las variables; por eso, es común normalizar o estandarizar los datos antes de aplicar PCA.
 - **Interpretabilidad:** Las componentes principales son combinaciones lineales de variables originales, lo que puede dificultar su interpretación directa.
-

***Nota:** Estos conceptos son fundamentales para entender cómo PCA transforma los datos y por qué es útil en visualización y preprocesamiento, temas que abordaremos en las siguientes unidades.*

Aplicación práctica y relevancia en la industria

En sectores como retail y e-commerce, PCA se utiliza para:

- **Visualizar datos complejos:** Reduciendo dimensiones a 2 o 3 para graficar y detectar patrones o segmentos de clientes.
- **Preprocesamiento:** Simplificar datos antes de aplicar algoritmos de clustering o clasificación, mejorando rendimiento y reduciendo ruido.

Por ejemplo, un analista puede usar PCA para transformar variables de comportamiento de compra en componentes principales que resumen tendencias clave, facilitando la segmentación de clientes.

Además, entender la varianza explicada permite justificar cuántos componentes usar, balanceando precisión y simplicidad.

Esta base teórica te prepara para implementar PCA en Python y analizar sus resultados, integrando visualizaciones y evaluaciones que veremos en las próximas unidades.

6 - Panorama de métodos: clustering, PCA y reglas de asociación

Este contenido introduce las técnicas fundamentales de aprendizaje no supervisado: clustering (K-Means, jerárquico, DBSCAN), reducción de dimensionalidad con PCA y reglas de asociación (Apriori). Se explican sus principios básicos, aplicaciones prácticas, limitaciones y decisiones clave para su uso en proyectos reales de ciencia de datos.

Introducción

Imagina que tienes un conjunto de datos sin etiquetas, como registros de compras en un e-commerce o sensores de una fábrica, y quieres descubrir patrones ocultos sin una guía previa. ¿Cómo identificar grupos naturales en los datos, reducir su complejidad para visualizarlos o encontrar reglas que expliquen comportamientos frecuentes? Estas preguntas son el corazón del aprendizaje no supervisado, una rama esencial en ciencia de datos que permite extraer conocimiento sin depender de etiquetas o resultados conocidos.

En esta unidad, exploraremos un panorama general de las técnicas más utilizadas: clustering (agrupamiento), reducción de dimensionalidad con PCA y reglas de asociación. Aprenderás qué problemas resuelven, cómo funcionan a nivel conceptual, sus limitaciones y cómo elegir la técnica adecuada según el contexto. Este conocimiento te preparará para implementar y analizar estos métodos en datos reales, un paso clave en tu formación como científico de datos.

Al finalizar, podrás relacionar cada técnica con casos concretos y entenderás las decisiones de diseño que influyen en su desempeño y resultados.

Panorama de técnicas de aprendizaje no supervisado

Clustering: agrupando datos similares

El **clustering** es una técnica que busca agrupar observaciones similares en conjuntos llamados *clústeres*, basándose en medidas de similitud o distancia. Es útil para segmentar clientes, detectar patrones en imágenes o agrupar documentos.

Principales métodos de clustering

- **K-Means:** Divide los datos en k grupos, asignando cada punto al centroide más cercano. Es eficiente y fácil de interpretar, pero requiere definir k y asume clústeres esféricos y de tamaño similar.
- **Clustering jerárquico:** Construye una jerarquía de clústeres mediante fusiones o divisiones sucesivas. No requiere definir k inicialmente y permite visualizar dendrogramas, pero puede ser costoso computacionalmente.
- **DBSCAN:** Agrupa puntos densamente conectados, identificando clústeres de forma arbitraria y detectando ruido. Requiere definir parámetros como *eps* (distancia máxima) y *min_samples* (mínimos puntos para formar un clúster).

***Nota:** La elección del método depende del tipo de datos, forma esperada de los clústeres y presencia de ruido.*

Reducción de dimensionalidad: PCA

La **reducción de dimensionalidad** busca representar datos de alta dimensión en espacios de menor dimensión, preservando la estructura relevante. Esto facilita la visualización, mejora el rendimiento de modelos y reduce ruido.

Análisis de Componentes Principales (PCA)

PCA transforma las variables originales en nuevas variables ortogonales llamadas *componentes principales*, que capturan la mayor varianza posible. Los primeros componentes resumen la información más importante.

- **Aplicaciones:** Visualización de datos complejos, preprocesamiento para clustering o clasificación.
- **Limitaciones:** Asume relaciones lineales y puede perder interpretabilidad.

Reglas de asociación

Las **reglas de asociación** extraen relaciones frecuentes entre ítems en conjuntos de transacciones, como productos comprados juntos en retail.

- **Algoritmo Apriori:** Identifica conjuntos frecuentes de ítems y genera reglas con métricas como soporte, confianza y lift.
- **Aplicaciones:** Recomendaciones de productos, análisis de cesta de la compra, optimización de inventarios.

***Importante:** Estas reglas no implican causalidad, solo co-ocurrencia.*

Decisiones de diseño y parámetros clave

Técnica	Parámetros clave	Consideraciones principales
K-Means	Número de clústeres k	Elegir k adecuado; sensible a valores atípicos
Clustering jerárquico	Método de enlace (single, complete)	Interpretación dendrograma; escalabilidad
DBSCAN	eps , $min_samples$	Detecta ruido; adecuado para formas arbitrarias
PCA	Número de componentes a conservar	Balance entre reducción y pérdida de información
Apriori	Soporte mínimo, confianza mínima	Controla cantidad y calidad de reglas generadas

Limitaciones y supuestos básicos

- Clustering asume que la similitud/diferencia entre puntos es significativa y que los datos pueden agruparse claramente.
- PCA asume linealidad y que la varianza es una medida adecuada de información.
- Reglas de asociación requieren datos transaccionales y pueden generar muchas reglas irrelevantes sin filtros adecuados.

Reflexiona: ¿Qué pasaría si aplicas K-Means a datos con clústeres de formas muy irregulares? ¿O PCA a datos con relaciones no lineales?

Aplicaciones prácticas y escenarios de uso

Estas técnicas son pilares en proyectos reales de ciencia de datos en sectores como retail, e-commerce, tecnología y analítica de negocios:

- **Clustering:** Segmentación de clientes para campañas personalizadas, detección de fraudes agrupando comportamientos atípicos, análisis de patrones en sensores industriales.

- **PCA:** Visualización de datos complejos para entender tendencias, reducción de ruido antes de aplicar modelos supervisados, compresión de datos para almacenamiento eficiente.
- **Reglas de asociación:** Identificación de productos que se compran juntos para promociones cruzadas, optimización de layout en tiendas físicas o virtuales, análisis de comportamiento de compra.

En la práctica, la elección y parametrización de cada método requiere entender el contexto del negocio, la naturaleza de los datos y los objetivos del análisis. Por ejemplo, en un e-commerce con datos ruidosos y formas de clústeres complejas, DBSCAN puede ser más adecuado que K-Means.

Este panorama te permitirá tomar decisiones informadas y preparar el terreno para implementar estas técnicas en notebooks, explorando sus resultados y ajustes.

Te pasamos un ejemplo para que te orientes en como se utiliza PCA para mejorar el rendimiento del modelo.

```
# %% [markdown]
# # ✓ Demostración controlada: PCA *mejora* el rendimiento:#
# En este ejemplo creamos un escenario didáctico donde:
# - Tomamos el dataset de cáncer de mama (scikit-learn, 30 features reales)
# - Añadimos *muchas* columnas de ruido (features irrelevantes)
# - Usamos K-Nearest Neighbors (KNN), un clasificador sensible al "curse of dimensionality"
# - Comparamos rendimiento **SIN PCA** vs **CON PCA (reducción fuerte a 30 componentes)**
# -----
# %% [markdown]
# #
# □ Importar librerías necesarias
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
import numpy as np
import time
import warnings
warnings.filterwarnings("ignore")
# %% [markdown]
# #
# □ Preparar dataset y añadir mucho ruidoSEED = 42np.random.seed(SEED)
data = load_breast_cancer()
X_real = data.data # 30 features realesy = data.target
# Ajusta este valor para aumentar/disminuir efecto de PCAAn_noise = 300 # <--
# muchas columnas de ruido para empeorar el caso SIN PCAAn_samples = X_real.shape[0]
# Generar ruido gaussianoX_noise = np.random.normal(loc=0.0, scale=1.0,
size=(n_samples, n_noise))
# Concatenar features reales + ruidoX = np.hstack([X_real, X_noise])
print(f"Dimensiones originales (sin ruido): {X_real.shape[1]}")
print(f"Dimensiones con ruido añadidas: {X.shape[1]}")
```

```

# %% [markdown]
# #
# □ División train/test (estratificada para mantener balance de clases)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=SEED, stratify=y)
# %% [markdown]
# #
# □ Baseline SIN PCA (KNN, sensible al ruido y dimensionalidad)
from sklearn.pipeline import make_pipeline
# Pipeline: StandardScaler -> KNNpipe_full = make_pipeline(StandardScaler(),
KNeighborsClassifier(n_neighbors=5) # KNN elegido por sensibilidad al ruido)
start = time.time()pipe_full.fit(X_train, y_train)
pred_full = pipe_full.predict(X_test)
acc_full = accuracy_score(y_test, pred_full)
time_full = time.time() - start
# %% [markdown]
# #
# □ Aplicar PCA con reducción fuerte (por ejemplo, 30 componentes) + mismo
KNNn_components_pca = 30 # reducimos mucho (330 -> 30), probar otros
valores si querés
# Escalado previo a PCA es buena prácticascaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
pca = PCA(n_components=n_components_pca, random_state=SEED)
X_train_pca = pca.fit_transform(X_train_s)
X_test_pca = pca.transform(X_test_s)
print(f"\nDimensión después de PCA (n_components retenidos):
{X_train_pca.shape[1]}")
start = time.time()
knn_pca = KNeighborsClassifier(n_neighbors=5)knn_pca.fit(X_train_pca,
y_train)
pred_pca = knn_pca.predict(X_test_pca)
acc_pca = accuracy_score(y_test, pred_pca)
time_pca = time.time() - start
# %% [markdown]
# #
# □ Resultados comparativos (esperamos: acc_full < acc_pca)
print("\n--- RESULTADOS ---")
print("□ Accuracy SIN PCA: ", round(acc_full, 6))
print("□ Tiempo SIN PCA: ", round(time_full, 4), "segundos")
print("□ Accuracy CON PCA: ", round(acc_pca, 6))
print("□ Tiempo CON PCA: ", round(time_pca, 4), "segundos")
print("□ Reducción de dimensiones:", X_train.shape[1], "→",
X_train_pca.shape[1])
# %% [markdown]
# #
# □ Observaciones y recomendaciones
# - Elegimos **KNN** porque su performance empeora notablemente con muchas
features irrelevantes;
# en ese escenario PCA **mejora la relación señal/ruido** y suele elevar
accuracy.

```

Dimensiones originales (sin ruido): 30
Dimensiones con ruido añadidas: 330

Dimensión después de PCA (n_components retenidos): 30

--- RESULTADOS ---

◆	Accuracy SIN PCA:	0.853147
🕒	Tiempo SIN PCA:	0.0793 segundos
◆	Accuracy CON PCA:	0.916084
🕒	Tiempo CON PCA:	0.0122 segundos
📉	Reducción de dimensiones:	330 → 30